

Wealth Intelligence — Technical Blueprint

The intelligence layer between portfolio data and the Relationship Manager. A deterministic engine computes every signal from a private-banking book; a language model only *explains* those signals. This document is written from the code that exists in the repository — every architectural claim maps to a module under `wealth_intelligence/`.

Repository — `wj200/juliusbaer-wi` Interface — Streamlit RM workbench Engine — pure standard-library Python
Signals — 8 detectors · Σ severity² triage Tests — 21 passing Data — synthetic · 20 clients · 5 snapshots

- Python · stdlib core
- Streamlit · workbench
- Claude API · Opus · explanation only
- Docker · container deploy
- unittest · 21 tests
- event_log.csv · authoritative events

00 Contents

01	Executive summary & architecture	02	Technology stack	03	The dataset & ingestion
04	The signal engine — eight detectors	05	The look-through resolver	06	Grounded explanation
07	The RM workbench	08	Data-handling decisions	09	Trust, governance & compliance
10	Testing strategy	11	Deployment & folder structure	12	Roadmap
13	Risks, limitations & caveats	14	The scenario engine	15	Data model & joins
16	Security, operations & error handling	17	The explainer contract		

Sections 01–07 describe the system top to bottom; 08–09 are the parts a reviewer scrutinises hardest — the data traps handled correctly and the compliance posture; 10–13 cover how it is verified, shipped, and where it is honest about its limits; 14–17 are the deep specs — the forward-looking scenario engine, the data model, the operational and security envelope, and the exact contract the language model is held to.

01 Executive summary & system architecture

The problem. A Relationship Manager (RM) covers twenty private-banking clients across Singapore and Hong Kong. Existing tools are *descriptive* — valuations, allocations, performance — and leave the RM to manually interpret risk, market impact, tax and the next action. The opportunity is an intelligence layer that moves the RM from "*what does the portfolio look like?*" to "*what should I know, and what should I do next?*"

The approach. Everything the system asserts is computed from the source files by a deterministic engine. Each signal it raises carries its severity, a one-line reason an RM could say aloud, the exact numbers behind it, and the source rows as evidence. Only after that does a language model act — and it is handed those findings as *facts to explain*, never as a problem to solve. That boundary is the whole compliance argument: no insight an RM cannot defend, and no number a model invented.

The result. A book-wide triage that answers the challenge's central question — *who does the RM call first, and can she defend the ranking?* — drilling into each client's story with a grounded explanation, and turning every insight into an action the RM accepts, edits or dismisses.

INPUT
12 files

External data · 12 synthetic source files

holdings · transactions · instruments · credit_facilities · mandates · portfolios
clients · commitments · planned_cash_needs · market_context · event_log · rm_notes

↓ join · convert to USD

PHASE 0
model

Book

joined · currency-aware · indexed

Look-through resolver

stock + perpetual + note → one issuer

↓ client_360 → detectors

PHASE 1
engine

Signal engine · deterministic detectors

Collateral

LTV vs call

Concentration

look-through

Mandate

bands · excl.

Liquidity

net of pledge

Income

draw vs yield

Attribution

event_log

Findings → Book triage · rank clients by Σ severity² · every finding carries facts + evidence

----- GOVERNANCE BOUNDARY · deterministic above · the model only explains below -----

↓ explain · facts only

PHASE 2
explain

Grounded explanation · the only LLM in the system

Claude turns findings into a situation read, talking points and honest watch-outs —
handed only computed facts, client profile, RM notes and the cited events. Cached · offline-safe fallback.

↓ render

PHASE 3
workbench

RM workbench · Streamlit

Book triage

Client story + “Why?”

Accept · Edit · Dismiss

↓ decide

Relationship Manager · Priscilla Ong

reviews, decides, and owns the advice

↓ Client advisory conversation

Fig 1 — the pipeline. The book descends one path. The eight detectors compute every signal and rank the twenty clients; the dashed line is the governance boundary. Only below it does a language model act. Remove the model and the engine still produces the entire triage — the page just loses its narration.

02 Technology stack

LAYER	CHOICE	WHY
Signal engine	Pure Python standard library	No third-party runtime dependency — runs anywhere a bank permits Python, trivial to audit, nothing to <code>pip install</code> .
Explanation	Claude (Anthropic API), <code>claude-opus-5</code>	Only for turning computed findings into language; adaptive thinking; keyed, cached, and never on the critical path for the numbers.
Workbench	Streamlit	Python-native UI over the engine's JSON — fastest path to the three-zone RM interface.
Tests	<code>unittest</code> (stdlib)	21 regression tests pin the flagship behaviours; no external runner needed.
Deploy	Docker · Streamlit Community Cloud	A container image for a bank-realistic deploy; Community Cloud for a public demo URL.

03 The dataset & ingestion

Twelve synthetic files describe one RM's book: 20 clients, 24 portfolios and 1,015 positions at **five dated snapshots** (Dec 2025 → Aug 2026) spanning a Middle East energy shock, a Strait of Hormuz closure, a June technology drawdown and a rate re-pricing. One snapshot tells you what a portfolio *is*; comparing snapshots tells you what *happened*.

FILE	GRAIN	WHAT IT CARRIES
holdings	1,015 rows	Every position at all five snapshots — the centre of gravity. USD value is authoritative per row.
instruments	62	Price history, <code>underlying_reference</code> (look-through), and the <code>sustainability_excluded / concentration_limit_applies</code> flags.
credit_facilities	5	Lombard / term loans with per-snapshot LTV and margin-call triggers.
mandates	48	Allocation bands, single-position limits, and binding sustainable exclusions.
transactions	393	Trades, income (coupons/dividends/distributions), fees, capital calls, facility activity.
planned_cash_needs	20	Known and expected future liabilities, with certainty and timing.
event_log	16	Authoritative 2026 events with the transmission channels that reached portfolios.
rm_notes	28	The RM's own notes — subjective, and often in tension with the numbers. Where advice lives.

Also: clients, portfolios, commitments, market_context (23 series × 5 dates, incl. ten FX pairs).

04 The signal engine — eight detectors

Each detector is a pure function `(Book, client_id) → list[Finding]`. Every number is computed from the source files; a detector never guesses, and is explicit where the data is ambiguous. A `Finding` carries a **severity** (Info · Low · Medium · High · Severe), a one-line headline, a plain-language detail, a `facts` dict (the exact numbers), and an `evidence` list (the source rows). The book-wide engine runs all eight across the twenty clients and ranks them by **Σ severity²** — one genuine emergency outranks a pile of minor drift — with ties broken by the single most-severe finding, then AUM. The full logic per detector:

1 · Collateral `detect_collateral`

Inputs: `credit_facilities.csv` — drawn, lending value, the margin-call trigger, and the LTV at each of the five snapshots. **Logic:** loan-to-value is read as `drawn ÷ lending_value` (lending value is already advance-rate haircut — not recomputed); the detector measures `distance = margin_call - LTV_now` and reads the whole 5-snapshot trail to flag a *rising* line (LTV up ≥ 3pts from the first snapshot). **Severity:** `distance ≤ 1.5 → Severe · ≤ 4 → High · ≤ 8 → Medium · else rising → Low · else silent`. **Fires on** CL-0014 (69.4% vs a 70% call → 0.6pt, Severe) and CL-0002 (73.7% vs 75%).

2 · Concentration `detect_concentration`

Inputs: today's holdings, `instruments` (underlying_reference, concentration_limit_applies), the mandate single-position limit. **Logic:** each holding is resolved through §05 to the single issuer it really carries risk to; the values are aggregated *across every portfolio the client holds*, keeping **managed** and **custody** value separate. A name is flagged when it exceeds the tighter of the mandate limit or 10% of the book. A limit *breach* can exist only in the managed portion; a custody-only stake is framed as "a concentration to discuss, not a breach to correct." **Severity:** `custody-only → High if ≥ 25% else Medium; otherwise over-limit → High (Severe if ≥ 25% and over-limit) · ≥ 15% → Medium · else Low`. **Fires on** CL-0014 (Golden Harbour ≈ 29% via stock + perpetual + accumulator, Severe), CL-0013 (Helios 22.5% once the ELN is counted), CL-0001 (Bara Nusantara 41%, custody legacy).

3 · Mandate & exclusions `detect_mandate`

Inputs: managed portfolios only (custody is not measured against a mandate), `mandates.csv` bands + single-position limit + notes, and `instruments.sustainability_excluded`. **Logic:** per managed portfolio, each asset-class weight is checked against its [min, max] band (breach at `> max + 0.5` or `< min - 0.5`) → Medium; for a sustainable mandate (notes carry binding exclusions), any held instrument flagged `excluded = Y` → High. The copy distinguishes drift

from a client-directed position (check the notes / any waiver). **Fires on** CL-0005 (an energy fund + a palm-oil name inside a Sustainable Balanced mandate).

4 · Liquidity `detect_liquidity`

Inputs: holdings (liquidity_tier), facilities (drawn), `planned_cash_needs` (certainty, currency, timing), `commitments` (uncalled). **Logic:** liquid = Daily + Weekly tiers, then **net the drawn facility** (a prior claim) → genuinely free liquidity; needs = confirmed / likely cash needs inside an 18-month horizon (converted to USD) + uncalled private-market commitments; coverage = free ÷ needs. A currency mismatch (obligation not in the liquid currencies) and gated funds are flagged. **Severity:** coverage < 1 → High · < 1.5 → Medium · gated & < 3 → Low. **Fires on** CL-0014 (USD 19m "liquid" → USD 12m free once the loan is netted) and CL-0017 (three-quarter-gated private credit + USD 15.8m uncalled).

5 · Income & drawdown suitability `detect_income_suitability`

Inputs: recurring consumption in `planned_cash_needs`, income in `transactions` (coupons, dividends, distributions, deposit interest), fixed-income unrealised P&L and maturities in holdings, and the client's life stage and horizon. **Logic:** annual draw vs the income the book actually paid (annualised from the observed quarters); coverage = income ÷ draw. Two honest branches — (A) coverage < 0.9 is a genuine *shortfall*, funded by selling capital (High if < 0.6 else Medium); (B) coverage ≥ 0.9 but the client draws on a bond book *below cost* with maturities *beyond their horizon* is a capital-quality flag (High if retired else Medium) — the README's flagship case, framed as a conversation, not a cash-flow alarm. **Fires on** CL-0006 (income covers only 25% of a USD 5m/yr draw) and CL-0012 (income covers the draw, but he is 71, retired, drawing on bonds due 2045 vs a ~2038 horizon).

6 · Event attribution `detect_attribution`

Inputs: holdings at the year-end baseline vs today (instruments held at both ends), and `event_log.csv` (primary_transmission). **Logic:** the largest USD declines per instrument are matched to the most-severe event whose transmission channel keyword maps to the holding's sector / asset class / region. Severity is capped at Medium — this is an *explanation*, not an alarm. **Fires on** the 5 Jun tech drawdown, whose channel explicitly names "collateralised lending", tying it to the collateral pressure on CL-0002 and CL-0014.

7 · Currency mismatch `detect_currency_mismatch`

Inputs: holdings (instrument_ccy), `planned_cash_needs` (currency, amount), and the FX series in `market_context`. **Logic:** obligations are grouped by currency; a currency is "mismatched" when the client owes materially in it (≥ USD 1m or 5% of the book) but holds < 20% of the book in it — meaning other assets must be converted at the exchange rate of the day. The YTD move of the lead obligation currency is stated directionally. **Severity:** ≥ 15% of book →

High else Medium. **Fires on** CL-0001 (an SGD 9m property deposit due against a largely USD book).

8 · **KYC / periodic review** `detect_review_due`

Inputs: `clients.kyc_review_due` vs today (2026-08-26). **Logic:** days to due — overdue → High · ≤ 45 days → Medium · ≤ 90 days → Low · else silent. An operational-compliance nudge so advice sits on an up-to-date file. **Fires on** CL-0011 (5 days), CL-0008 (16), CL-0014 (27), CL-0004 (35), and others.

05 The look-through resolver

A structured product's asset class tells you what it is *called*; its `underlying_reference` tells you what you are *exposed to*. The resolver maps a holding to the single issuer it really carries risk to, so concentration can be measured the way a risk officer would — counting a stock, its perpetual and an accumulator on it as *one* bet.

- **Direct** — the holding *is* the name (a stock, that issuer's perpetual, or a single-underlying note) → one exposure.
- **Basket** — a worst-of / basket note → one exposure per named constituent, flagged so it is not silently merged into the headline single-name number.
- Diversified funds, sovereigns, deposits and gold → no single-name exposure.

The mapping is an **explicit, auditable alias table** — every entry can be pointed at the source field that justifies it. That is the governance posture: no free-association about what a note "probably" references.

Worked example — CL-0014, Lau Chi Ming. Golden Harbour appears three ways: the stock, a 5.25% perpetual, and a 12-month accumulator. Individually each looks modest; rolled up to the name they are $\approx 29\%$ of the managed book — and the accumulator sits in "double-up below strike" territory while his Lombard LTV is 0.6 points from a margin call.

06 Grounded explanation

The engine decides *what is true*. This layer turns a client's Findings into language the RM can use: a situation summary, two to four talking points / actions to consider, and an honest list of what to check. The model is handed **only** the computed facts, the client's profile and objectives, the RM notes, and the cited event rows — and is instructed to reason strictly from them.

The prompt contract

- Reason only from the supplied facts, findings, notes and events. Never introduce a number, holding or world event that is not in the input.
- Every claim must be defensible to the client in a meeting — prefer "we should check X" over a confident assertion the data does not support.
- The RM remains responsible: suggest actions "to consider", never instructions.
- Return a strict JSON object: `situation`, `talking_points`, `watch_outs`.

Robustness

Every explanation is **cached to disk**, keyed by a hash of its exact inputs, so the hero clients can be pre-generated once and the demo runs with **no network**. If there is no API key or any error, the layer falls back to a deterministic explanation assembled from the Findings themselves — the app always works.

Model: `claude-opus-5` with adaptive thinking. The key is read from the environment (or Streamlit Secrets) and never committed. A refusal or failure degrades to the deterministic path.

07 The RM workbench

A single Streamlit surface in three zones, matching *signal* → *understanding* → *decision*:

- **Book** — the triaged book in the sidebar: who to call first, each with a severity chip and a one-line reason.
- **Client** — the selected client's grounded explanation, plus a look-through exposure snapshot.
- **Signals** — every finding with a **"Why?"** expander showing the exact facts and source rows, and **Accept / Edit / Dismiss** so the RM stays in control.

It opens on the highest-priority client (Lau Chi Ming) so the first frame already shows what the system does.

08 Data-handling decisions

The dataset sets traps; handling them correctly is much of the assessment.

DECISION	RATIONALE	WHAT IT COSTS
----------	-----------	---------------

Value from <code>market_value_usd</code>, never the base-currency AUM columns	Avoids reading a HKD→USD conversion as a loss — Lau's book "falls" 206m→26m only on paper.	None — it is the correct column.
LTV read as <code>drawn / lending_value</code>	Lending value is already advance-rate haircut, per the data dictionary — not raw market value.	None.
Custody separated from managed	A client-directed legacy stake is a concentration to discuss, not a mandate breach to correct.	More logic per finding.
Encumbered collateral netted from "sellable" liquidity	Pledged assets are a prior claim; a book that looks liquid can be anything but.	Requires joining facilities to holdings.
Income annualised from observed quarters, with the estimate labelled	Honest about a run-rate rather than asserting a precise annual figure.	A caveat the RM must read.

09 Trust, governance & compliance

AI-driven advisory must preserve trust and the central role of the RM. The architecture answers each requirement structurally, not with a disclaimer.

- **Explainability** — every insight is computed by a named detector and carries its facts; the "Why?" expander shows them.
- **Suitability** — findings respect mandate, risk profile and objectives; the model is told to as well.
- **Human oversight** — the RM accepts, edits or dismisses every insight; nothing is auto-actioned.
- **Traceability** — each finding lists its source rows as evidence (e.g. `credit_facilities.csv:CF-0002`).
- **Compliance** — the model never computes a number or free-associates about the world; explanations are grounded in the authoritative event log.
- **Security** — no client data leaves the engine; the API key is read from the environment and never committed; the explanation layer sends only synthetic, already-computed facts.

10 Testing strategy

21 standard-library tests pin the behaviour the demo depends on — run with `python -m unittest discover -s tests:`

- **Loading & FX** — book shape (20/24/1,015/62/393) and USD conversion for HKD and EUR.

- **Collateral** — Lau's margin call is SEVERE and <1pt from the line; a comfortable facility stays quiet.
- **Concentration** — Golden Harbour rolls up ≥ 3 instruments and breaches the limit; a custody legacy stake is *not* a breach.
- **Mandate** — the sustainable-exclusion breach fires for CL-0005.
- **Liquidity** — encumbered collateral is netted (free < gross).
- **Income** — the retiree flags impaired capital (not a false shortfall); a genuine shortfall flags; "Pre-retirement" is not treated as retired.
- **Ranking** — Lau tops the book; all 20 clients analyse without error.

11 Deployment & folder structure

```
juliusbaer-wi/
├── wealth_intelligence/      # the engine – pure standard library
│   ├── data_model.py        # Phase 0 · joined, currency-aware Book
│   ├── lookthrough.py       # Phase 0 · positions → single issuer
│   ├── findings.py          # severity + Finding types
│   ├── detectors.py         # Phase 1 · eight detectors (+ detectors_extra)
│   ├── engine.py            # Phase 1 · run + rank the book
│   ├── explainer.py         # Phase 2 · grounded NL (Claude)
│   └── cli.py / __main__.py  # CLI triage
├── streamlit_app.py          # Phase 3 · RM workbench
├── scripts/pregenerate.py    # warm the explanation cache
├── tests/test_engine.py      # 21 unittests
├── data/                    # 12 synthetic source files
├── docs/                    # architecture.html · blueprint.html · DATA_DICTIONARY.md
├── requirements.txt · runtime.txt · Dockerfile
└── .streamlit/config.toml    # navy/gold theme + secrets bridge
```

Run it

- **Local** — `pip install -r requirements-app.txt && streamlit run streamlit_app.py`
- **Docker** — `docker build -t jb-wi . && docker run -p 8080:8080 -e ANTHROPIC_API_KEY=... jb-wi`
- **Streamlit Cloud** — point it at `streamlit_app.py`, add `ANTHROPIC_API_KEY` in Secrets, deploy.

For the demo, warm the cache first: `python scripts/pregenerate.py CL-0014 CL-0012 CL-0017` — then it runs with no network.

12 Roadmap

- **Shipped since v1** — the scenario toggle (\$14) and the currency-mismatch

- **Scenario toggle** — "Middle East de-escalates / worsens", re-pricing exposures from the market series to show forward risk.
 amp; KYC-review detectors are now built.
- **Meeting pack export** — a one-page prep sheet per client the RM can take into the room.
- **More detectors** — currency-mismatch on future liabilities, tax-lot harvesting across a household, KYC-review-due surfacing.
- **Right-sizing for scale** — the stdlib model holds twenty clients in memory; a real book moves to a columnar store behind the same detector interface.

13 Risks, limitations & caveats

- **Income is a run-rate estimate.** Annualised from the observed quarters of transactions; the detector says so rather than asserting a precise figure.
- **Look-through is a curated map.** It is auditable but must be maintained as the instrument universe changes; an unrecognised note contributes no single-name exposure rather than a wrong one.
- **Private-market valuations lag.** Quarterly-reported funds are a quarter behind by design — a feature of the asset class, surfaced honestly.
- **The engine finds only what it encodes.** The model cannot surface a signal the detectors do not compute — a deliberate trade for auditability over open-ended inference.
- **Synthetic data throughout.** No real client, instrument or identifier is present; the habit of treating it as real is part of the exercise.

14 The scenario engine — "what could happen next?"

The five snapshots answer *what happened*. The scenario engine (`scenarios.py`) adds the forward-looking half of the brief's advisory flow — *what could happen next* — as a toggle in the workbench: **De-escalation** · **Today** · **Escalation** of the unresolved 2026 Middle East situation.

Mechanism

- **Shock vector.** Each scenario is a small, transparent set of market moves whose direction and rough magnitude match what the snapshots actually contain — Escalation: Brent +30%, gold +10%, HK property −12%, UST +40bp; De-escalation the mirror.
- **Per-instrument sensitivity.** A holding's price change is derived from its own attributes — `sector` · `region` · `asset_class` · `underlying_reference` — via an explicit sensitivity table

- (energy up, HK property down, long-duration bonds down on higher yields, gold up, tech down, cash flat).
- **Re-derivation.** Every holding is re-priced, then the figures the RM cares about are recomputed: book value change, each facility's LTV (and whether it now crosses the margin call), and single-name look-through exposure.
 - **Re-ranking.** The book re-orders by a scenario stress score (a margin breach dominates, then drawdown), so the queue answers "who is newly at risk?".

The demo moment. Under Escalation, Lau Chi Ming's LTV moves 69.4% → **70.4%** — across the margin call — and Ravi's 73.7% → 78.1%; both surface to the top of the queue. Flip to De-escalation and Hartono's energy-heavy book gains.

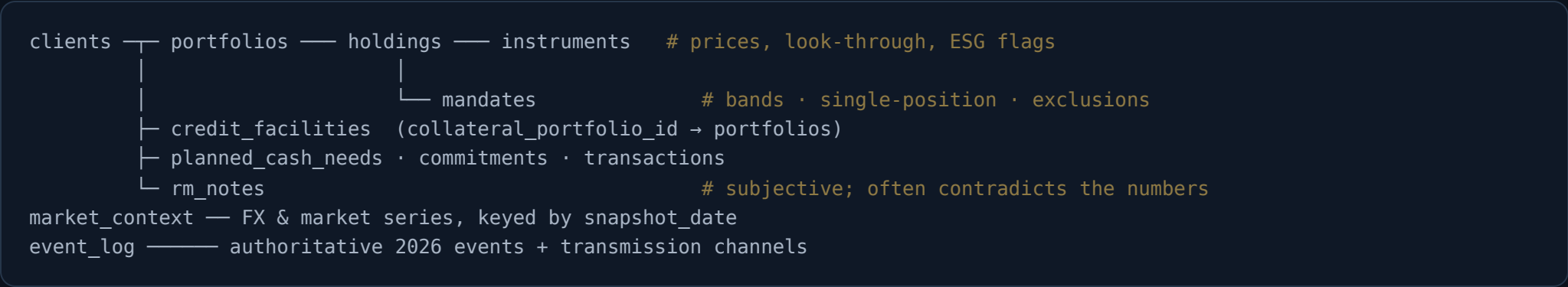
Honesty

It is a **stylised sensitivity model, not a full risk engine**, and labelled as such on screen with its shock vector visible. Crucially it stays inside the governance boundary: the shocks and sensitivities are computed in `scenarios.py` and shown to the RM — never produced by a language model. Two regression tests pin the flagship behaviour (Lau breaches under Escalation; the book recovers under De-escalation).

15 Data model & joins

All twelve files load into one in-memory `Book` (`data_model.py`) — pure standard library, indexed for the access patterns the detectors need. Twenty clients is small enough to hold whole in memory and read by hand.

The join graph



Currency

Every value is reduced to USD from `holdings.market_value_usd` at each snapshot — never the base-currency `aum_<date>` columns. The FX table (ten pairs × five snapshots) drives conversion

of cash needs and drawn facilities, with each pair's quote direction (per-USD vs USD-per) handled explicitly.

Access surface

The Book exposes `holdings_of · portfolios_of · facilities_of · notes_of · cash_needs_of · transactions_of · total_usd` and a `client_360()` that assembles everything about one client — the substrate every detector reads.

16 Security, operations & error handling

- **No data egress.** The engine runs locally; client data never leaves it. Only the already-computed findings, the client profile and the cited event rows are sent to the explanation model — and only when a key is present.
- **Key handling.** The API key is read from the environment (or Streamlit Secrets, bridged to the environment) and never written to disk or committed; `.env` and `.wi_cache/` are git-ignored.
- **Fault isolation.** The engine wraps each detector: a fault in one becomes an Info-level "could not run" finding rather than sinking the client's whole analysis.
- **Graceful degradation.** No key or no network → the explainer returns a deterministic summary assembled from the findings; the app is fully functional offline.
- **Determinism & caching.** Explanations are cached to disk keyed by a hash of their exact inputs, so they regenerate when the inputs change and never drift silently.
- **Operability.** A stdlib engine plus one container image (`Dockerfile`, non-root runtime port, healthcheck) runs on any bank-approved Python host; the UI is served by Streamlit or as a static page.

17 The explainer contract

The Phase-2 layer (`explainer.py`) is the only component that calls a language model, and it is held to a strict contract.

ASPECT	CONTRACT
Model	<code>claude-opus-5</code> , adaptive thinking; key from the environment.
Input	Only the computed findings (facts), the client profile & objectives, the RM notes, and the cited event rows — assembled by <code>build_context()</code> .
Instruction	Reason strictly from the supplied facts; introduce no number, holding or world event not in the input; prefer "we should check X" over an unsupported assertion; suggest actions "to consider", never

	instructions.
Output	A strict JSON object — <code>situation</code> (3–4 sentences), <code>talking_points</code> (2–4), <code>watch_outs</code> (1–3). Parsed leniently; a malformed reply falls back.
Refusal / error	A refusal or any exception degrades to the deterministic summary; the app never blocks on the model.
Caching	Keyed by <code>sha256(context + model + prompt version)</code> ; pre-generate the hero clients before a demo so it runs with no network.

The result is the compliance guarantee in one line: **the model can produce no number, because it is never given the job of producing one.**